

LAB 5- Capturing application metrics with Istio service mesh.

Lab Task 1 – Basic Kubernetes operations

I provisioned my cluster using the provided provisioned script with the name as shown.

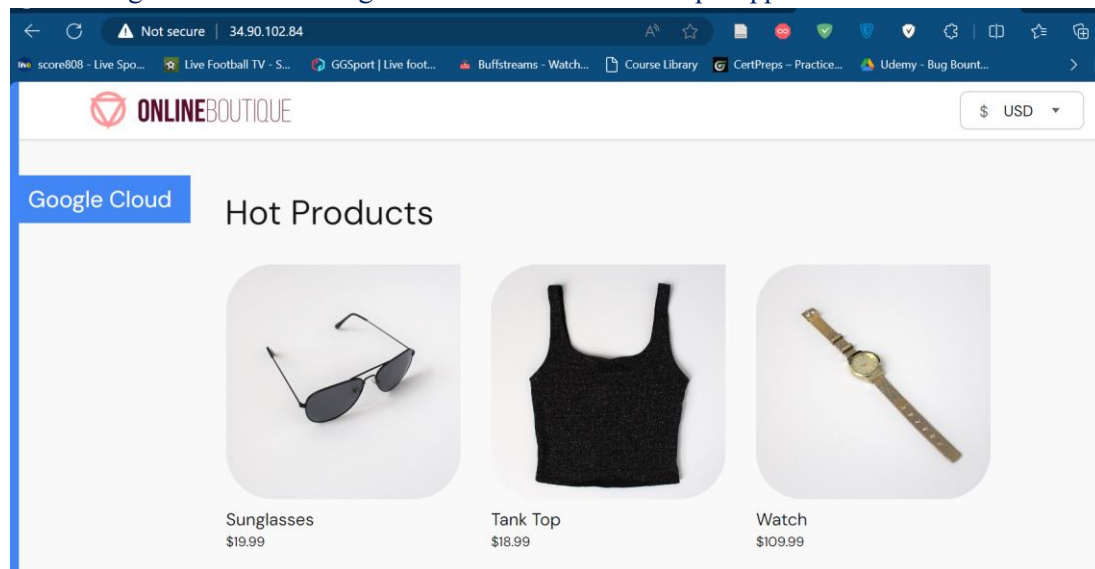
```
oadevusi@oluSec:~/aiops/Assignments/LAB5$ ./provision2.sh onlineboutique aiops4 europe-west4

No current clusters found, continuing to deploy one
Note: The Kubelet readonly port (10255) is now deprecated. Please update your workloads to use the recommended alternatives.
engine/docs/how-to/disable-kubelet-readonly-port for ways to check usage and for migration instructions.
Note: Your Pod address range (--cluster-ipv4-cidr) can accommodate at most 1008 node(s).
Creating cluster onlineboutique in europe-west4-a... Cluster is being health-checked (Kubernetes Control Plane is healthy)...
Created [https://container.googleapis.com/v1/projects/aiops4/zones/europe-west4-a/clusters/onlineboutique].
To inspect the contents of your cluster, go to: https://console.cloud.google.com/kubernetes/workload/_gcloud/europe-west4-a/o
kubecfg entry generated for onlineboutique.
NAME          LOCATION    MASTER VERSION  MASTER IP    MACHINE TYPE  NODE VERSION  NUM NODES  STATUS
onlineboutique europe-west4-a 1.30.5-gke.1014001 34.90.212.79 e2-standard-4 1.30.5-gke.1014001 1          RUNNING
```

I then ran **kubectl get svc** to get the services running:

```
oadevusi@oluSec:~/aiops/Assignments/LAB5$ kubectl get svc
NAME                                TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
adservice                          ClusterIP      34.118.235.132 <none>         9555/TCP         8m11s
cartservice                        ClusterIP      34.118.227.234 <none>         7070/TCP         8m10s
checkoutservice                   ClusterIP      34.118.232.175 <none>         5050/TCP         8m10s
currencyservice                   ClusterIP      34.118.236.161 <none>         7000/TCP         8m10s
emailservice                       ClusterIP      34.118.229.147 <none>         5000/TCP         8m9s
frontend                          ClusterIP      34.118.227.98  <none>         80/TCP           8m9s
istio-gateway-istio               LoadBalancer  34.118.238.117 34.90.102.84   15021:32250/TCP,80:31980/TCP 8m1s
kubernetes                        ClusterIP      34.118.224.1   <none>         443/TCP          11m
paymentservice                    ClusterIP      34.118.229.91  <none>         50051/TCP        8m8s
productcatalogservice              ClusterIP      34.118.230.86  <none>         3550/TCP         8m8s
recommendationservice              ClusterIP      34.118.228.55  <none>         8080/TCP         8m8s
redis-cart                         ClusterIP      34.118.235.120 <none>         6379/TCP         8m7s
shippingservice                    ClusterIP      34.118.227.118 <none>         50051/TCP        8m7s
```

Then navigated to the web to get the frontend of the boutique app.



Also ran the command **kubectl get svc -n istio-system** to get the services running in Istio namespace.

```
oadevusi@oluSec:~/aiops/Assignments/LAB5$ kubectl get svc -n istio-system
NAME                                TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
grafana                          ClusterIP      34.118.238.66  <none>         3000/TCP         13m
istio-egressgateway              ClusterIP      34.118.236.72  <none>         80/TCP,443/TCP   14m
istio-ingressgateway             LoadBalancer  34.118.238.248 34.32.197.215   15021:30113/TCP,80:31927/TCP,443:31080/TCP,31400:32281/TCP,15443:32663/TCP 14m
istiod                           ClusterIP      34.118.230.115 <none>         15010/TCP,15012/TCP,443/TCP,15014/TCP 14m
jaeger-collector                  ClusterIP      34.118.233.34  <none>         14268/TCP,14250/TCP,9411/TCP,4317/TCP,4318/TCP 13m
kiali                             ClusterIP      34.118.235.57  <none>         20001/TCP,9090/TCP 12m
loki                             ClusterIP      34.118.235.131 <none>         3100/TCP,9095/TCP 12m
loki-headless                    ClusterIP      None           <none>         3100/TCP         12m
loki-memberlist                   ClusterIP      None           <none>         7946/TCP         12m
prometheus                        ClusterIP      34.118.233.36  <none>         9090/TCP         12m
tracing                          ClusterIP      34.118.229.65  <none>         80/TCP,16685/TCP 13m
zipkin                           ClusterIP      34.118.231.75  <none>         9411/TCP         13m
```

Then fired up my Prometheus using istioctl dashboard prometheus
And when I ran

`rate(istio_requests_total{destination_canonical_service="productcatalogservice"}[1m])` query in Prometheus, I got duplicate entries because istio collects metrics from two different reporters for each request.

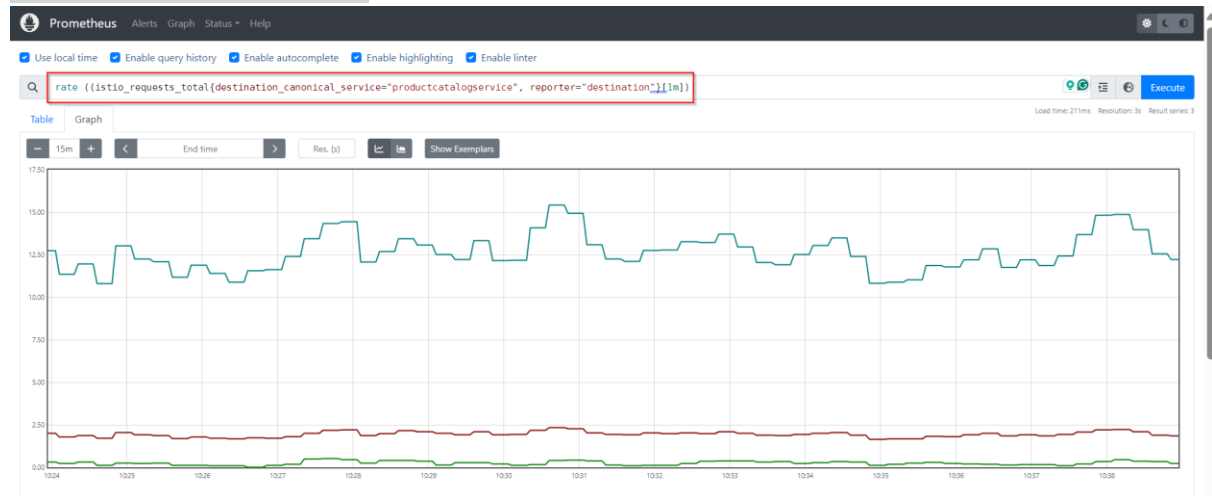
reporter="source": metrics reported by the istio proxy of the source service (where the request originated)

reporter="destination": metrics reported by the istio proxy of the destination service (where the request was received)

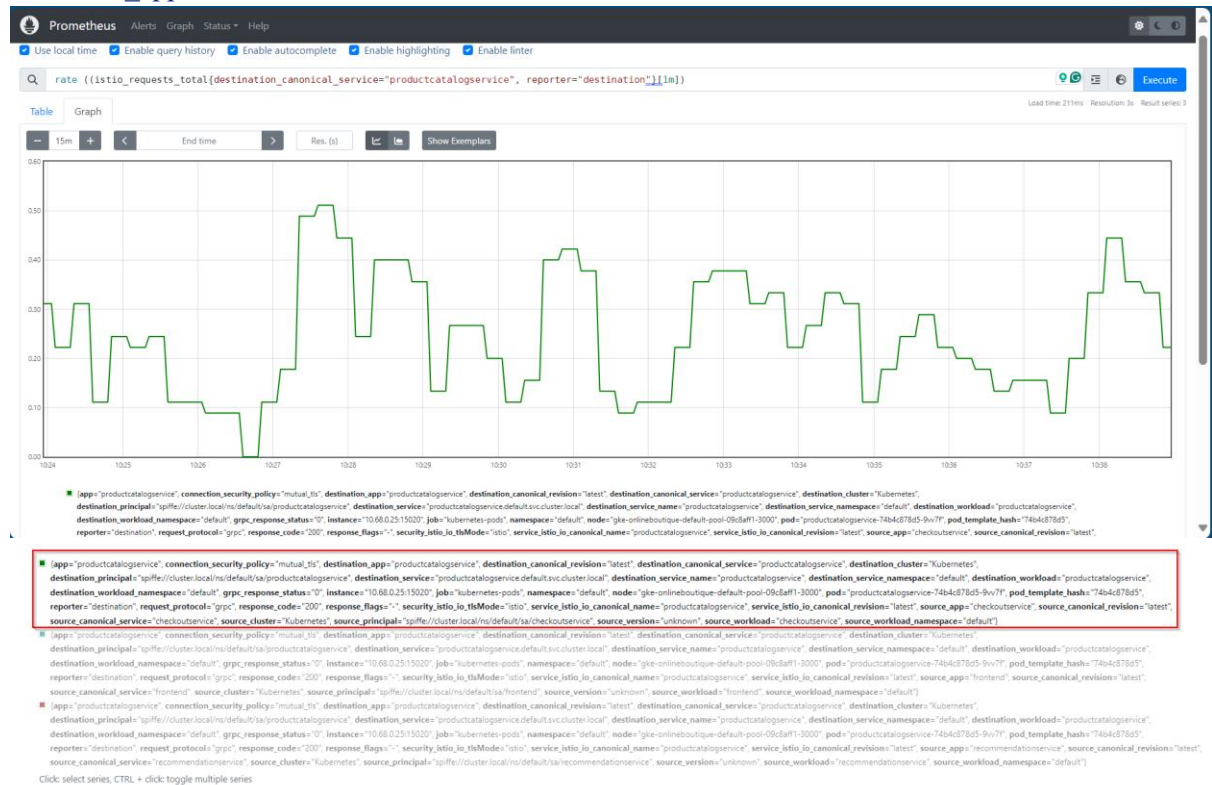
This dual reporting happens because each service in the mesh has its own istio proxy that reports metrics independently.

And filtering using the **destination**:

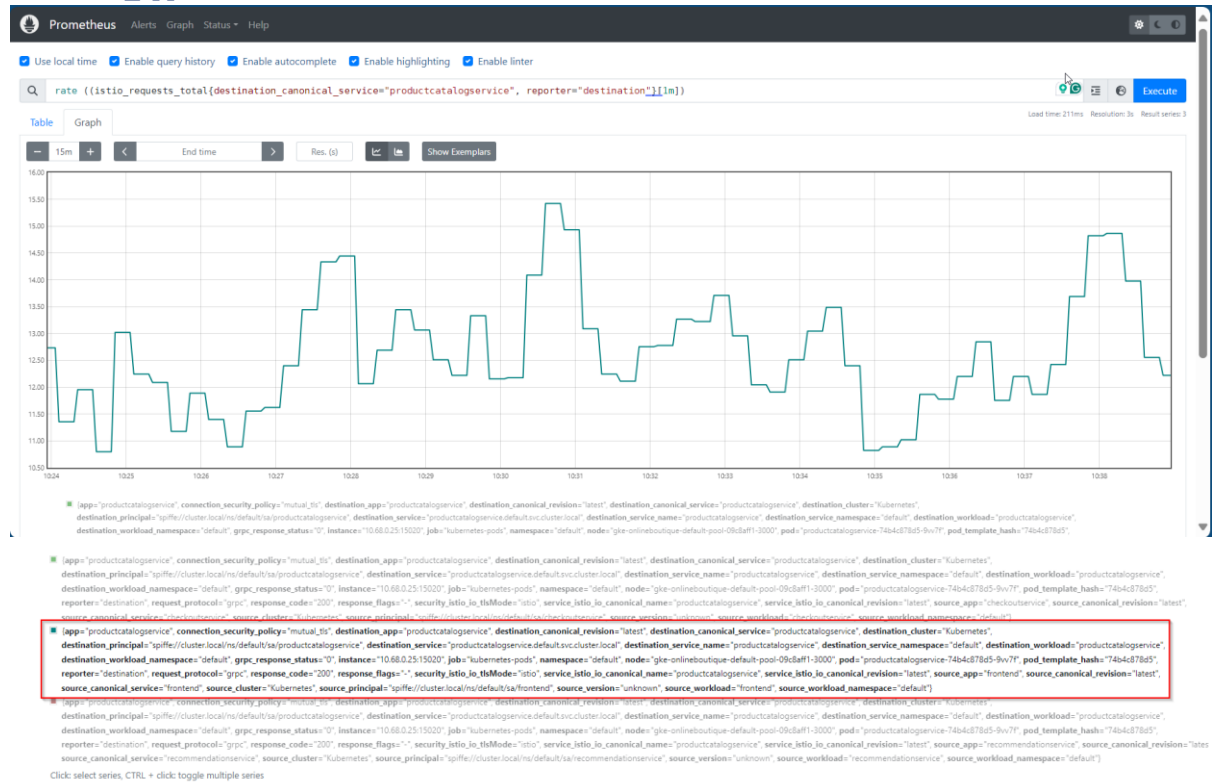
`rate(istio_requests_total{destination_canonical_service="productcatalogservice", reporter="destination"}[1m])`



For `source_app = checkoutservice`



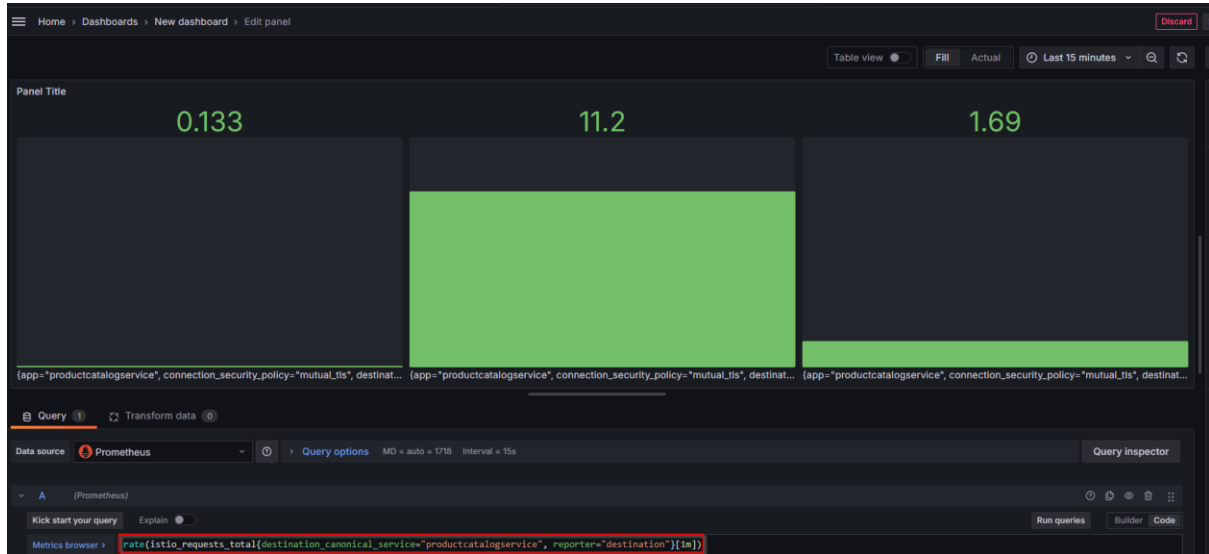
For source_app = frontend



For source_app = recommendationservice



Grafana dashboard showing the checkoutservice , frontend and recommendationservice

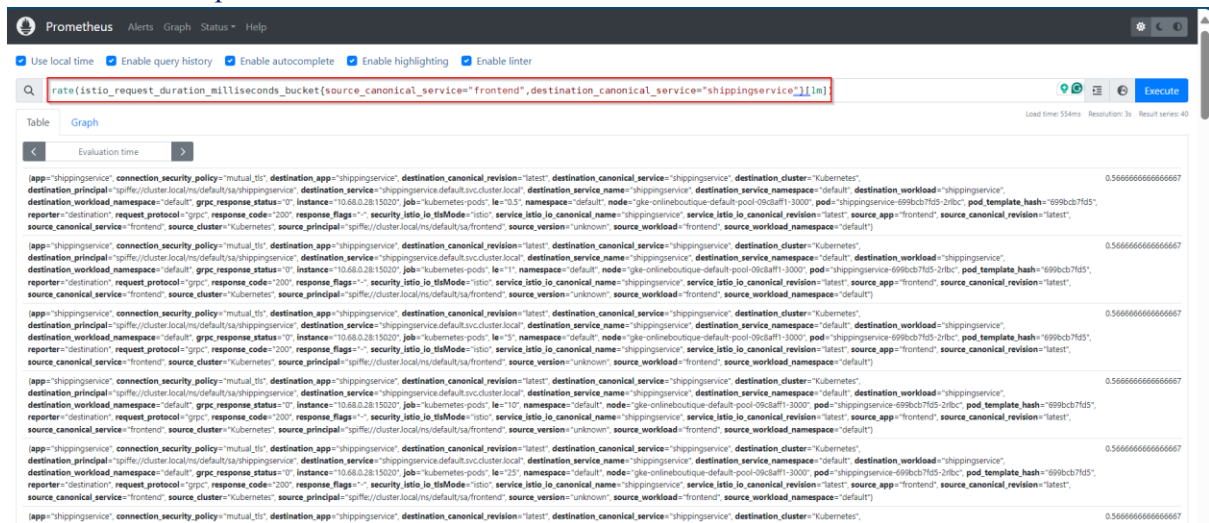


For the istio_request_duration_milliseconds_bucket creating histogram:

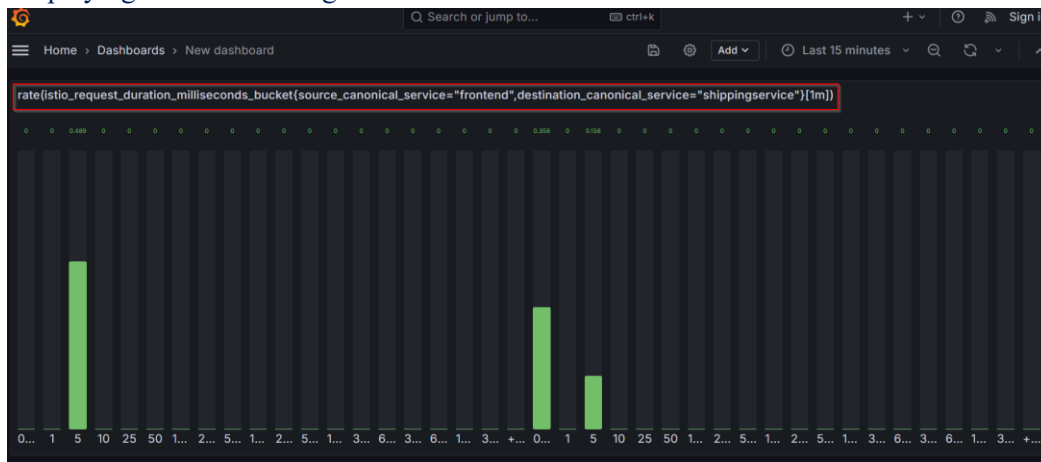
Before filtering I used:

```
rate(istio_request_duration_milliseconds_bucket{source_canonical_service="frontend", destination_canonical_service="shippingservice"})[1m]
```

The reason for duplication is also the same as above.

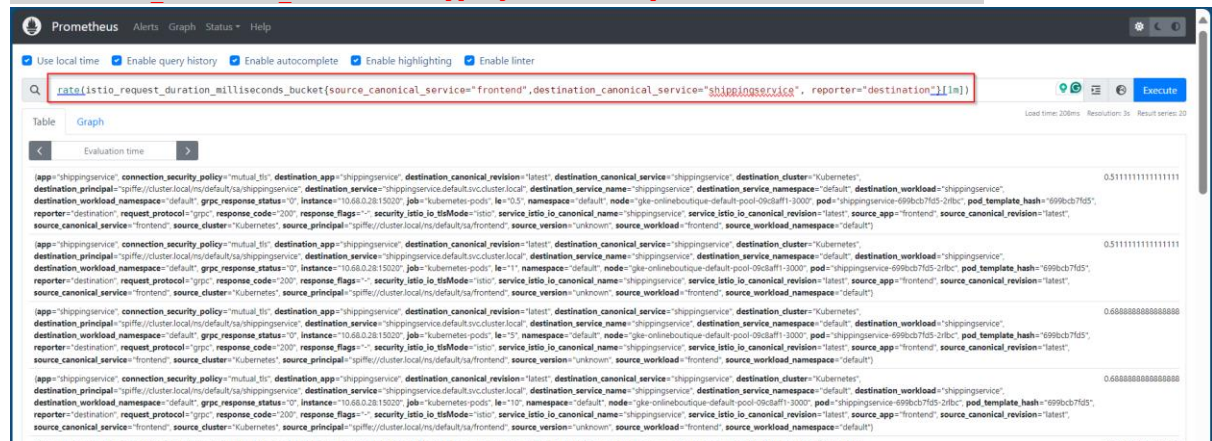


Displaying it on Grafana I got

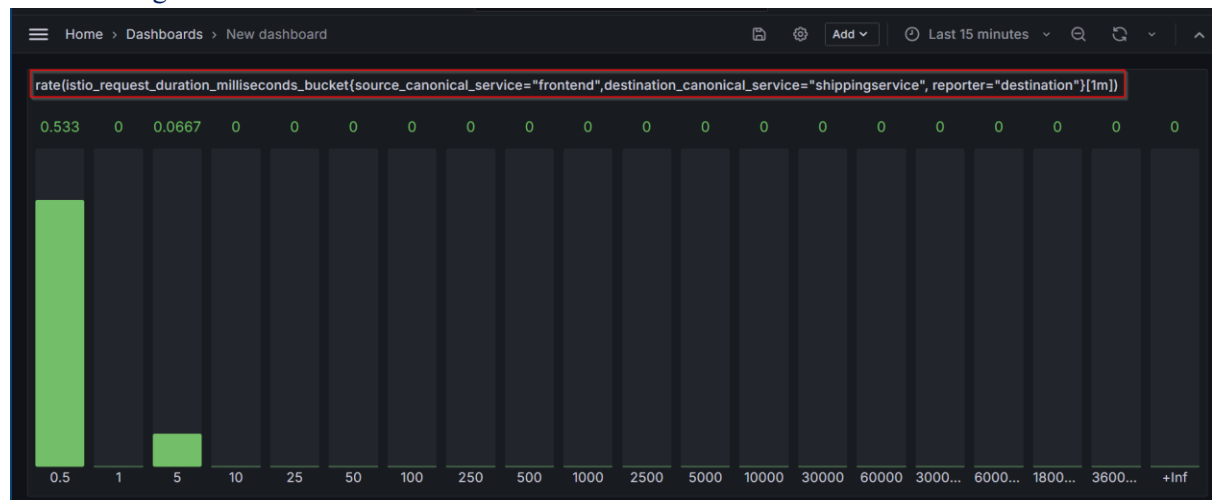


Now after filtering using:

```
rate(istio_request_duration_milliseconds_bucket{source_canonical_service="frontend",  
destination_canonical_service="shippingservice",reporter="destination"}[1m])
```

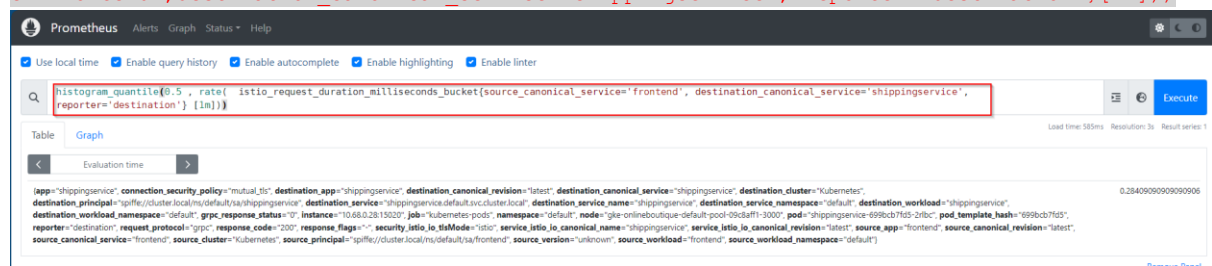


And showing the rate on the Grafana dashboard:



The Prometheus dashboard for the histogram_quantile at the 0.5 level using

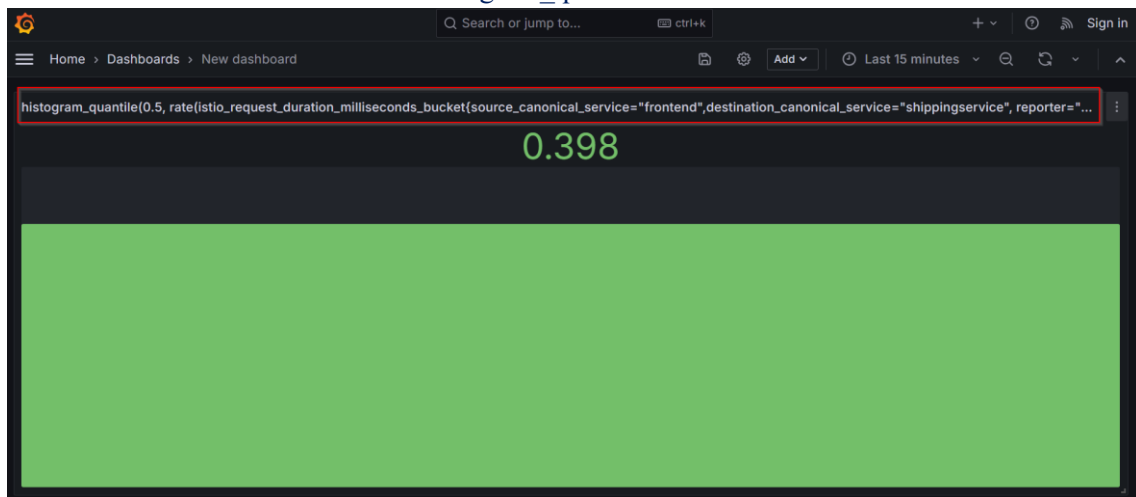
```
histogram_quantile(0.5,rate(istio_request_duration_milliseconds_bucket{source_canonical_servic  
e="frontend",destination_canonical_service="shippingservice", reporter="destination"}[1m]))
```



The Prometheus graph dashboard for the histogram_quantile at the 0.5 level



And the Grafana dashboard for the histogram_quantile at the 0.5 level

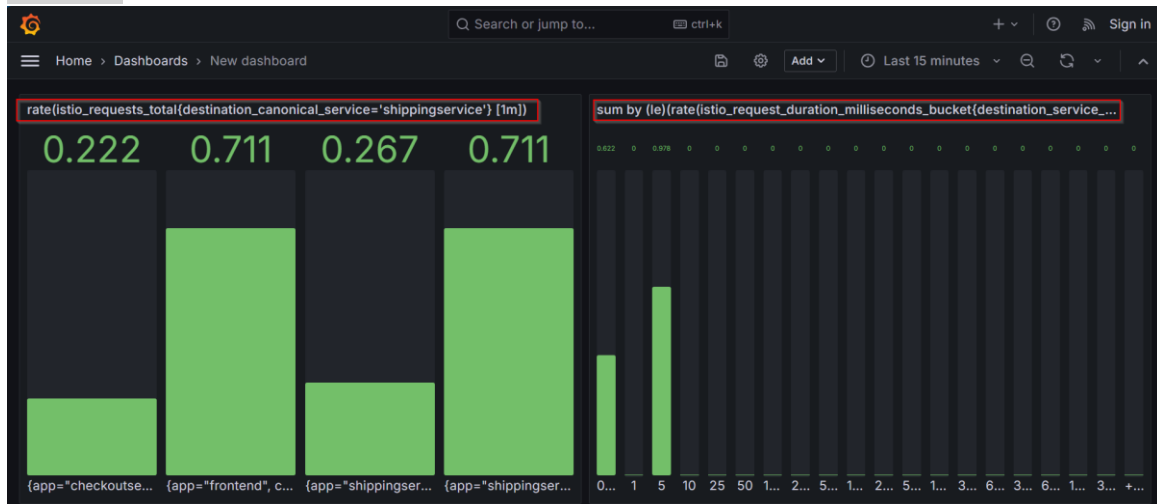


Screen shot of the two displays (request count time series and duration histogram) in Grafana
Using Request Count Time Series Panel:

`rate(istio_requests_total{destination_canonical_service='shippingservice'}) [1m]`

And for Request Duration Histogram Panel: `sum by`

`(le)(rate(istio_request_duration_milliseconds_bucket{destination_service_name="shippingservice"} [1m]))`



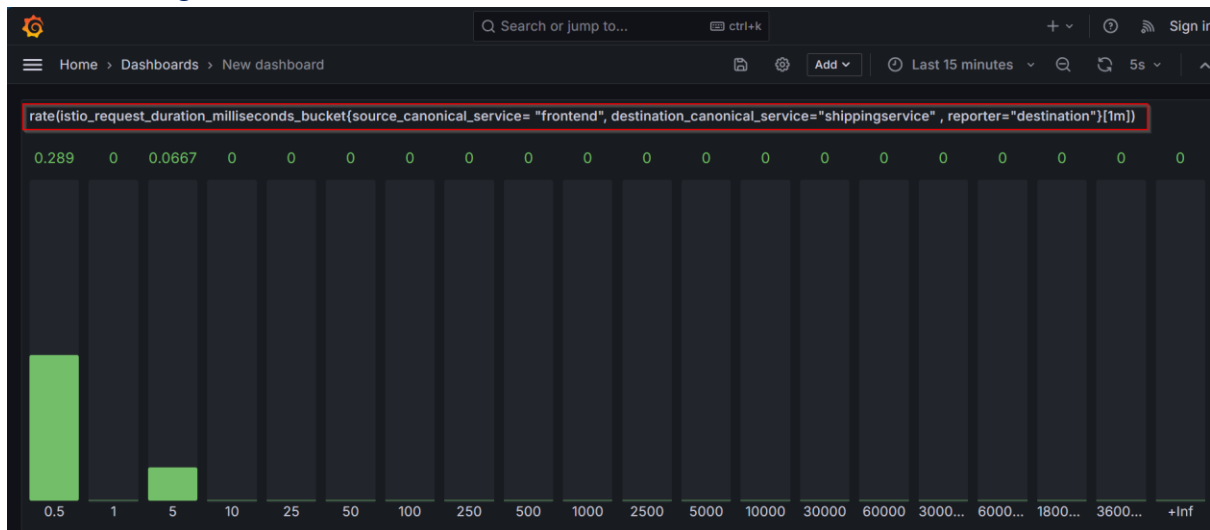
I scaled the loadgenerator to 2 replicas.

```
oadewusi@oluSec:~/aiops/Assignments/LAB5$ kubectl scale deployment loadgenerator --replicas=2
deployment.apps/loadgenerator scaled
oadewusi@oluSec:~/aiops/Assignments/LAB5$ kubectl get deploy
```

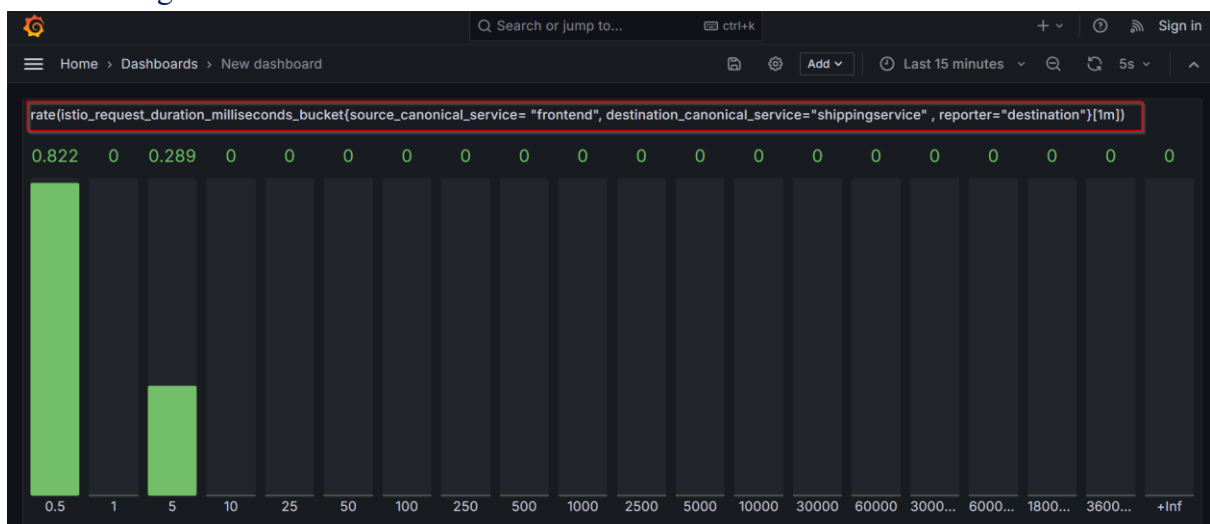
NAME	READY	UP-TO-DATE	AVAILABLE	AGE
adservice	1/1	1	1	163m
cartservice	1/1	1	1	163m
checkoutservice	1/1	1	1	163m
currencyservice	1/1	1	1	163m
emailservice	1/1	1	1	163m
frontend	1/1	1	1	163m
istio-gateway-istio	1/1	1	1	163m
loadgenerator	2/2	2	2	163m
paymentservice	1/1	1	1	163m
productcatalogservice	1/1	1	1	163m
recommendationservice	1/1	1	1	163m
redis-cart	1/1	1	1	163m
shippingservice	1/1	1	1	163m

After scaling to load generator to 2, I noticed the rate (especially for 0.5 bucket)increased as shown on the dashboard when compared to the previous rate.

Before scaling:



After scaling:

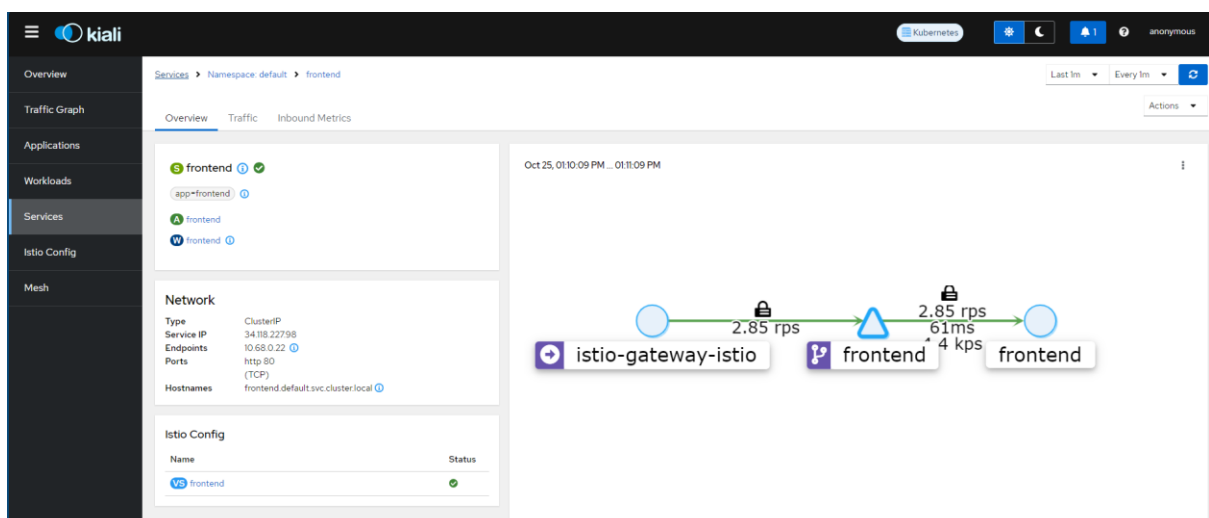
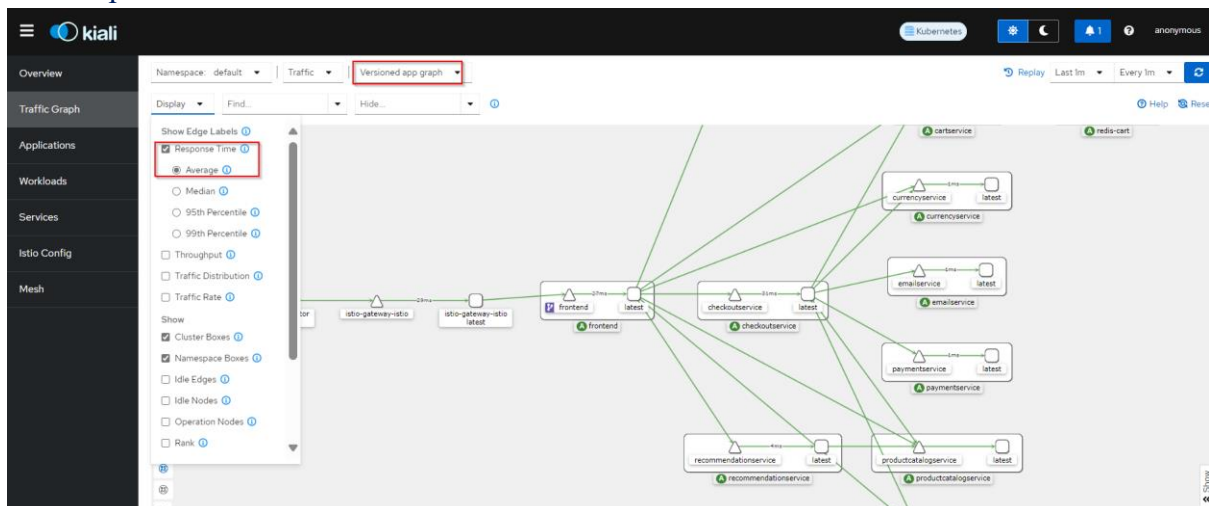


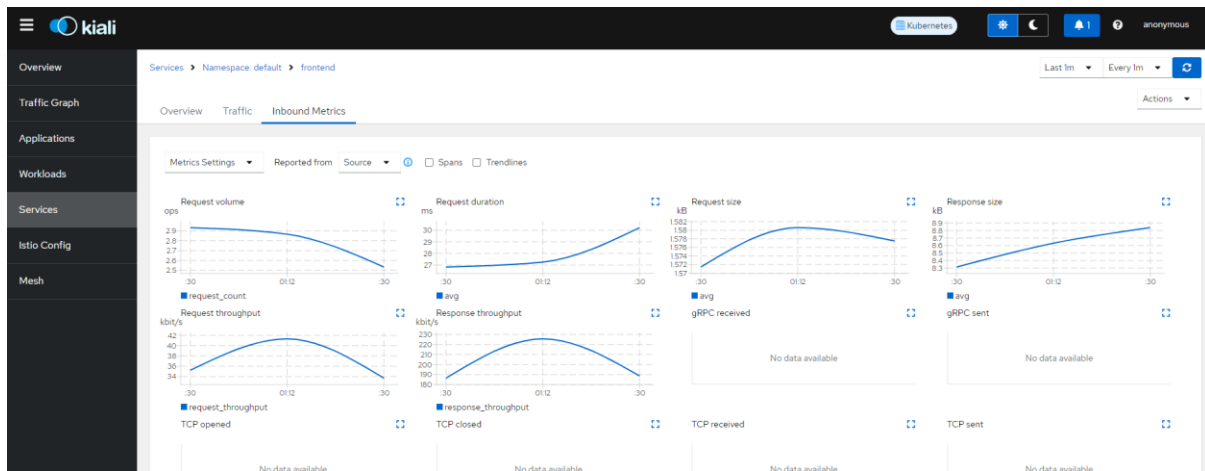
I then scaled it back to 1.

```
oadewusi@oluSec:~/aiops/Assignments/LAB5$ kubectl scale deployment loadgenerator --replicas=1
deployment.apps/loadgenerator scaled
oadewusi@oluSec:~/aiops/Assignments/LAB5$ kubectl get deploy
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
adservice            1/1     1             1           171m
cartservice          1/1     1             1           171m
checkoutservice      1/1     1             1           171m
currencyservice      1/1     1             1           171m
emailservice         1/1     1             1           171m
frontend             1/1     1             1           171m
istio-gateway-istio  1/1     1             1           171m
loadgenerator        1/1     1             1           171m
paymentservice       1/1     1             1           171m
productcatalogservice 1/1     1             1           171m
recommendationservice 1/1     1             1           171m
redis-cart           1/1     1             1           171m
shippingservice      1/1     1             1           171m
```

Lab Task 2 – Install the Kiali cluster visualization dashboard

Kiali exploration screenshots:





Lab Task 3 – Inject selective delay faults to the Boutique shipping service

Using **kubectl describe deployment frontend** to see the details of the deployment.

```
oadewusi@olusSec:~/aiops/Assignments/LAB5$ kubectl describe deployment frontend
Name: frontend
Namespace: default
CreationTimestamp: Wed, 23 Oct 2024 22:00:18 +0200
Labels: app=frontend
Annotations: deployment.kubernetes.io/revision: 1
Selector: app=frontend
Replicas: 1 desired | 1 updated | 1 total | 1 available | 0 unavailable
StrategyType: RollingUpdate
MinReadySeconds: 0
RollingUpdateStrategy: 25% max unavailable, 25% max surge
Pod Template:
  Labels: app=frontend
  Annotations: sidecar.istio.io/rewriteAppHTTPProbers: true
  Service Account: frontend
  Containers:
    server:
      Image: us-central1-docker.pkg.dev/google-samples/microservices-demo/frontend:v0.10.1
      Port: 8080/TCP
      Host Port: 0/TCP
      Limits:
        cpu: 200m
        memory: 128Mi
      Requests:
        cpu: 100m
        memory: 64Mi
      Liveness: http-get http://:8080/_healthz delay=10s timeout=1s period=10s #success=1 #failure=3
      Readiness: http-get http://:8080/_healthz delay=10s timeout=1s period=10s #success=1 #failure=3
  Environment:
    PORT: 8080
```

Yaml file to inject fault to app label: frontend:

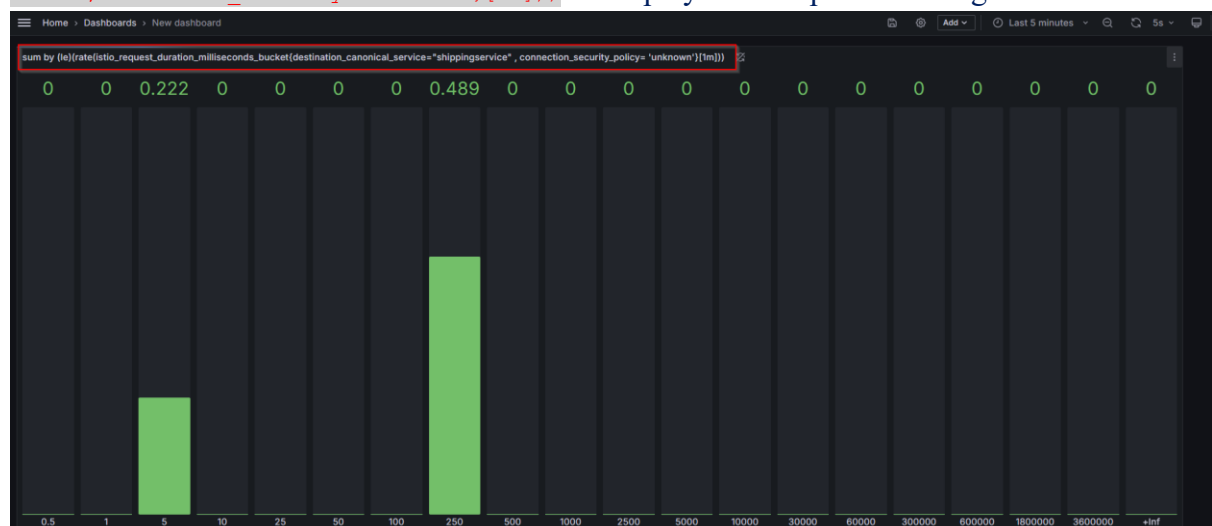
```
! shipping_delay_fromfront.yaml X
LAB5 > ! shipping_delay_fromfront.yaml
1  apiVersion: networking.istio.io/v1beta1
2  kind: VirtualService
3  metadata:
4    name: shippingservice
5  spec:
6    hosts:
7    - shippingservice
8    http:
9    - match:
10      - sourceLabels:
11        app: "frontend"
12      fault:
13        delay:
14          fixedDelay: .2s
15          percent: 100
16      route:
17        - destination:
18          host: shippingservice
19    - route:
20      - destination:
21        host: shippingservice
22
23
```

Applying the yaml file:

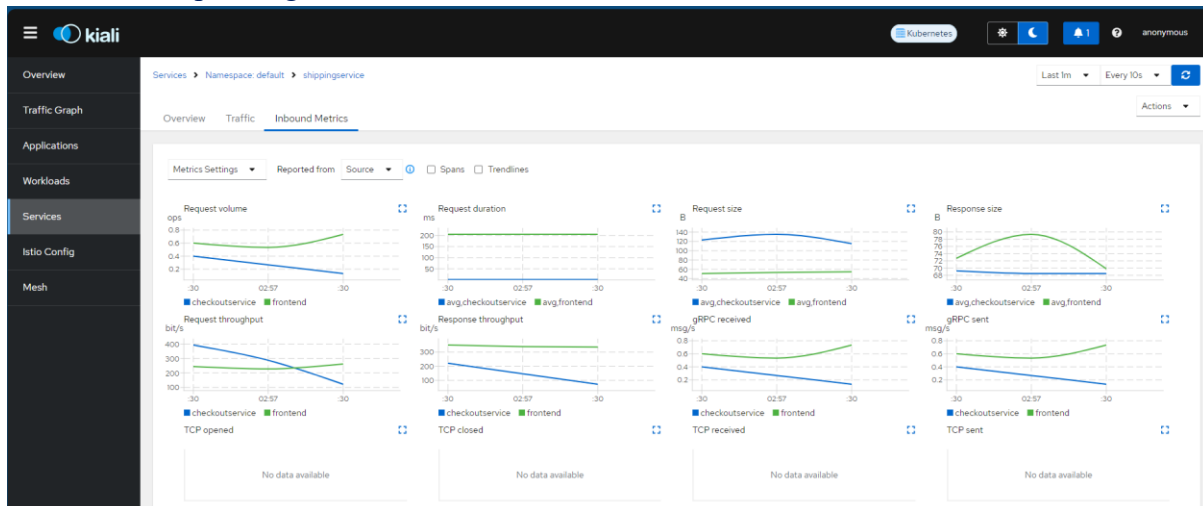
```
oadevus@oluSec:~/aiops/Assignments/LAB5$ kubectl apply -f shipping_delay_fromfront.yaml
virtualservice.networking.istio.io/shippingservice created
```

Using `sum by`

`(le) (rate(istio_request_duration_milliseconds_bucket{destination_canonical_service="shippingservice",connection_security_policy='unknown'}[1m]))` to display the two peaked histogram



And the corresponding data on Kiali:



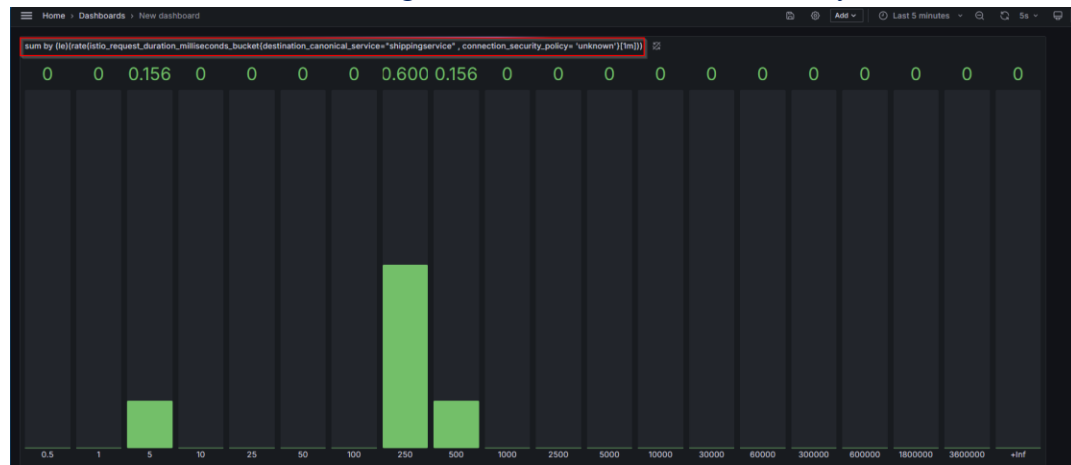
Adding a second rule to delay requests from checkoutservice by 0.4s that also match a specific URI with the string “GetQuote”.

```
... ! shipping_delay_pair.yaml X
LAB5 > ! shipping_delay_pair.yaml
1  apiVersion: networking.istio.io/v1beta1
2  kind: VirtualService
3  metadata:
4    name: shippingservice
5  spec:
6    hosts:
7      - shippingservice
8    http:
9      - match:
10         - sourceLabels:
11             app: "frontend"
12         fault:
13             delay:
14                 fixedDelay: 0.2s
15                 percent: 100
16         route:
17             - destination:
18                 host: shippingservice
19      - match:
20         - sourceLabels:
21             app: "checkoutservice"
22         uri:
23             regex: ".*GetQuote"
24         fault:
25             delay:
26                 fixedDelay: 0.4s
27                 percent: 100
28         route:
29             - destination:
30                 host: shippingservice
31      - route:
32         - destination:
33             host: shippingservice
```

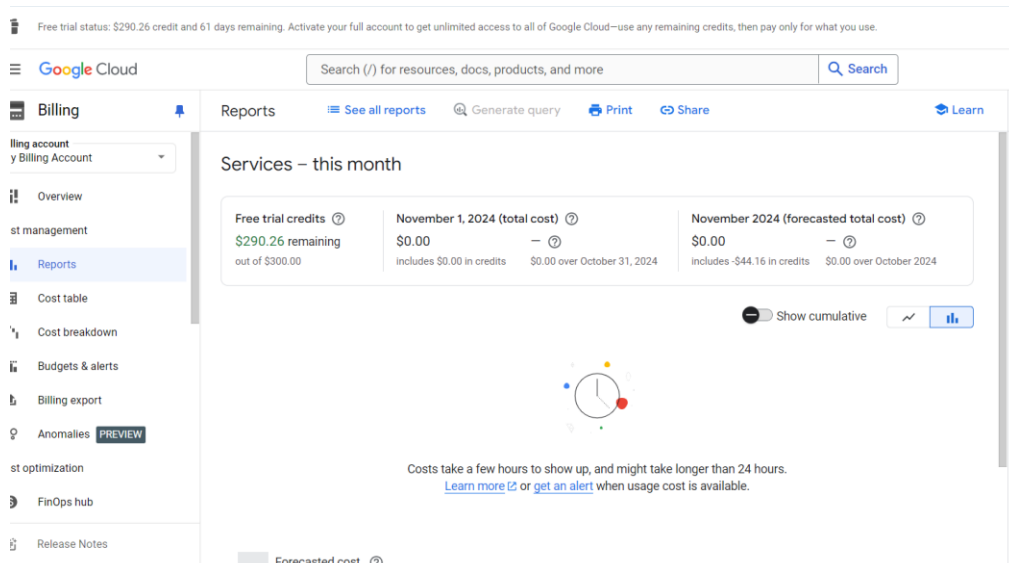
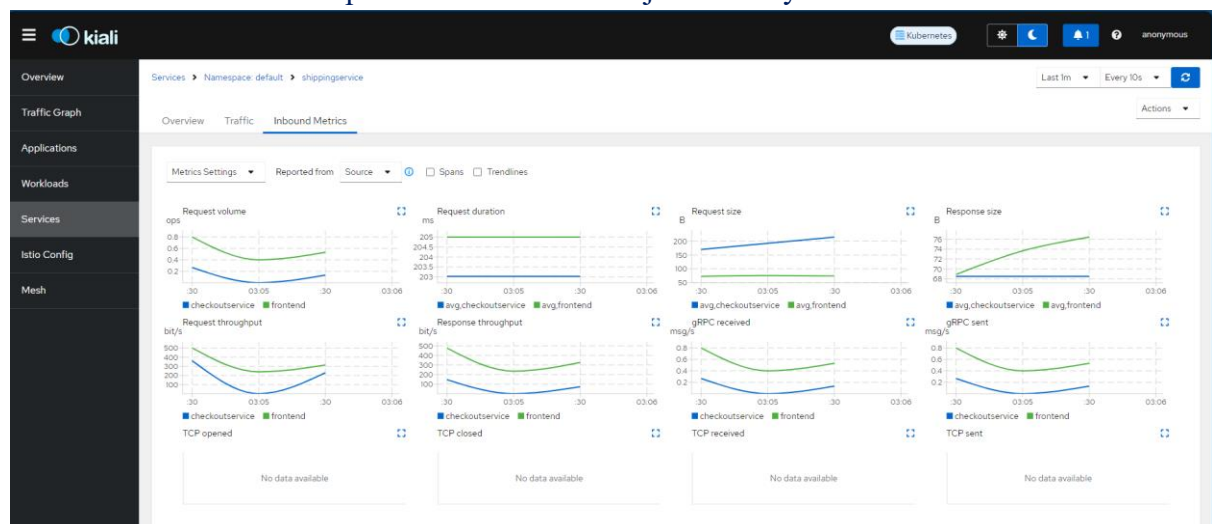
Applying the yaml file:

```
oadewusi@oluSec:~/aiops/Assignments/LAB5$ kubectl apply -f shipping_delay_pair.yaml
virtualservice.networking.istio.io/shippingservice created
```

Grafana visualization for histogram for both 0.4 and 0.2 sec delays



Kiali dashboard showing the details page for the shippingservice. Looking at the inbound metrics from the source reporter verification of injected delay



Free trial status: \$290.26 credit and 61 days remaining. Activate your full account to get unlimited access to all of Google Cloud—use any remaining credits, then pay only for what you use.

DISMISSACTIVATE

Google Cloud

aiops4

Search (/) for resources, docs, products, and more

Search

Compute Engine

Disks

CREATE DISKREFRESHDELETE

LEARN

Virtual machines

VM instances

Instance templates

Sole-tenant nodes

Machine images

TPUs

Committed use discounts

Reservations

Migrate to Virtual Machin...

Storage

Marketplace

Release Notes

Filter

Enter property name or value

Status

Name

Type

Size

Architecture

Zone(s)

In use by

Snapshot schedule

Actions

No rows to display

Disks

Every Compute Engine VM instance is attached to at least one disk as a boot disk (HDD) or a local SSD for high-

Now viewing project "aiops4" in organization "andrew.cmu.edu"